파이썬 완전 백과사전 2권 · 딥러닝 심화: 언어와 생성

2일차

어텐션부터 작은 GPT까지 처음부터 직접 만들기

6~10차시 · 책 12장 어텐션과 트랜스포머

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/12-transformer

오늘의 시간표

차시	주제	책 범위
6	어텐션: 부드럽게 꺼내는 딕셔너리	12장 들어가며 ~ 12.2
7	셀프 어텐션과 마스크	12.3 ~ 12.4
8	멀티헤드, 위치 정보, 블록 조립	12.5 ~ 12.7
9	작은 GPT 학습시키고 들여다보기	12.8 ~ 12.9
10	부품을 하나씩 빼 보기 실습, 12장 정리	12.10 ~ 12.11

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분

수업 전 준비: 12-transformer 폴더에서 b17_data.py(법전 내려받기)와 b20_train.py(작은 GPT 학습)를 한 번 실행해 둔다

6차시

06

어텐션: 부드럽게 꺼내는 딕셔너리

책 12장 들어가며 ~ 12.2절

학습 목표

- RNN과 트랜스포머가 문장을 읽는 방식의 차이를 말한다
- 쿼리·키·값으로 어텐션을 직접 계산하고 모양을 따라간다
- 점수를 d의 제곱근으로 나누는 이유를 숫자로 설명한다

들어가며 "그것"은 무엇을 가리킬까요?

고양이가 생선을 먹지 않았다. **그것**이 너무 **배불렀기** 때문이다. → 고양이

고양이가 생선을 먹지 않았다. **그것**이 너무 **상했기** 때문이다. → 생선

RNN (1일차)

한 글자씩 차례로 읽고 메모장(은닉 상태) 하나에 눌러 담아 넘긴다. 멀리 있는 정보는 흐려지고, 앞을 다 읽어야 다음을 계산한다

트랜스포머 (2017)

모든 글자가 다른 모든 글자를 한 번에 직접 본다. 어디를 얼마나 볼지는 데이터에서 배운다. 이 장치가 **어텐션**

12.1 키가 정확히 같지 않아도 꺼낸다면

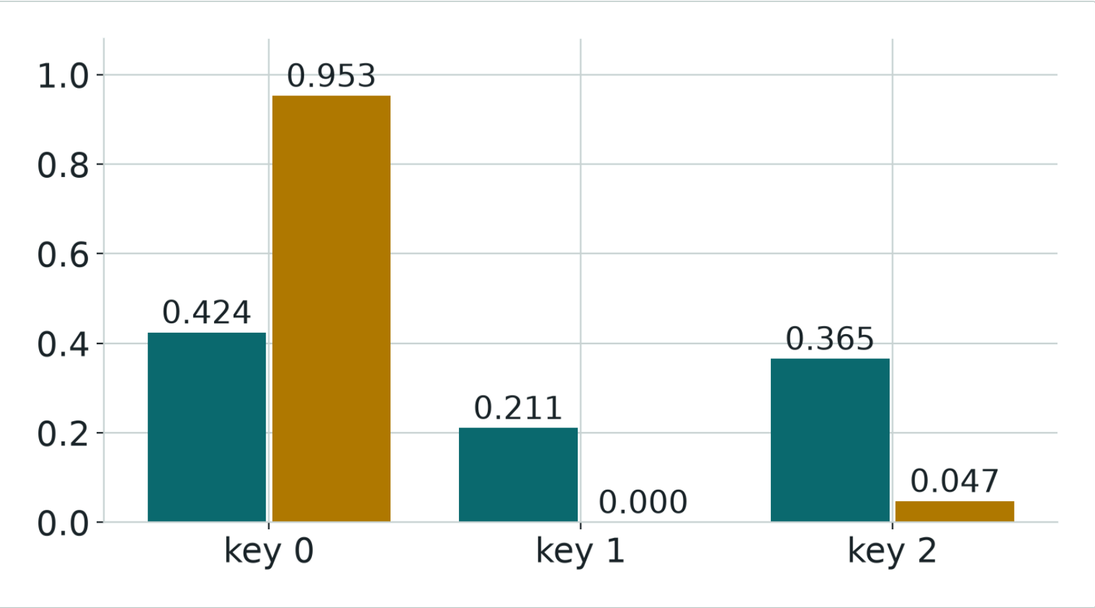
```
키 = torch.tensor([[0.9, 0.6],      # 떡볶이: (매운 정도, 단 정도)
                   [0.0, 1.0],      # 마카롱
                   [1.0, 0.1]])     # 김치찌개
값 = torch.tensor([4500.0, 2500.0, 9000.0])
질문 = torch.tensor([1.0, 0.5])     # "맵고, 조금 달았으면 좋겠어요"
점수 = 키 @ 질문                    # 내적 → 닮은 정도
가중치 = torch.softmax(점수, dim=0)  # 합이 1인 비율로
print("섞어 꺼낸 값:", (가중치 @ 값).round())
```

```
딕셔너리: 4500
점수   : tensor([1.200, 0.500, 1.050])
가중치: tensor([0.424, 0.211, 0.365])
섞어 꺼낸 값: tensor(5722.)
```

- 떡볶이 1.2 · 김치찌개 1.05 · 마카롱 0.5
- 메뉴판에 없는 **5,722원**: 닮은 만큼 섞은 값

6차시 · 12.1 어텐션: 부드럽게 꺼내는 딕셔너리 · 예제 12-transformer/b01_soft_dict.py (핵심 부분)

12.1 점수 차이를 키우면 딕셔너리가 된다



■ 점수 그대로 · ■ 점수 × 20 · key 0 떡볶이, 1 마카롱, 2 김치찌개

```
가중치 = torch.softmax(점수 * 20, dim=0)

20배 가중치: tensor([0.953, 0.000, 0.047])
  → tensor(4713.)
```

차이가 **크면** 하나를 콕 집어 꺼내고 (딕셔너리), 차이가 **작으면** 여러 개를 골고루 섞는다

그래서 "부드러운 딕셔너리"

12.1 어텐션의 세 재료

쿼리 q

손님의 주문

"나는 이런 것을 찾는다"

키 k

메뉴판에 적힌 특징

"나는 이런 특징을 가졌다". 쿼리와 내적해서 점수를 낸다

값 v

실제로 내주는 음식

가중치대로 섞여서 결과가 된다

주문(q)과 메뉴판(k)을 맞춰 보고 **점수** → 소프트맥스로 **비율** → 그 비율로 음식(v)을 **섞어** 내준다

12.1 질문 여러 개를 한꺼번에: 어텐션 함수

```
def 어텐션(q: torch.Tensor, k: torch.Tensor, v: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
    """질문 q마다 키 k와 닮은 정도를 재서, 그 비율대로 값 v를 섞는다."""
    점수 = q @ k.transpose(-2, -1) / math.sqrt(k.size(-1))   # (질문 수, 키 수)
    가중치 = 점수.softmax(dim=-1)                              # 질문마다 합이 1
    return 가중치 @ v, 가중치                                   # (질문 수, 값 차원)

q = torch.randn(2, 4)       # 질문 2개, 각각 숫자 4개
k = torch.randn(5, 4)       # 키 5개 (질문과 길이가 같아야 내적이 된다)
v = torch.randn(5, 3)       # 값 5개 (키와 개수가 같아야 한다), 각각 숫자 3개
출력, 가중치 = 어텐션(q, k, v)
```

```
가중치: (2, 5) 출력: (2, 3)
tensor([[0.022, 0.055, 0.039, 0.237, 0.648],
        [0.737, 0.069, 0.163, 0.004, 0.027]])
질문마다 가중치 합: tensor([1.000, 1.000])
```

- 이 **세 줄**이 이 장 전체의 심장
- 질문마다 보는 곳이 다르다: 4번 키, 0번 키

6차시 · 12.1 질문 여러 개를 한꺼번에 · 예제 12-transformer/b02_attention_fn.py (핵심 부분)

12.1 모양을 따라가 보기

계산	모양	뜻
q @ k.transpose(-2, -1)	(2, 4) @ (4, 5) → (2, 5)	질문마다 키 5개와의 점수
softmax(dim=-1)	(2, 5) 그대로	키 방향으로 합이 1
가중치 @ v	(2, 5) @ (5, 3) → (2, 3)	질문마다 섞은 값 하나

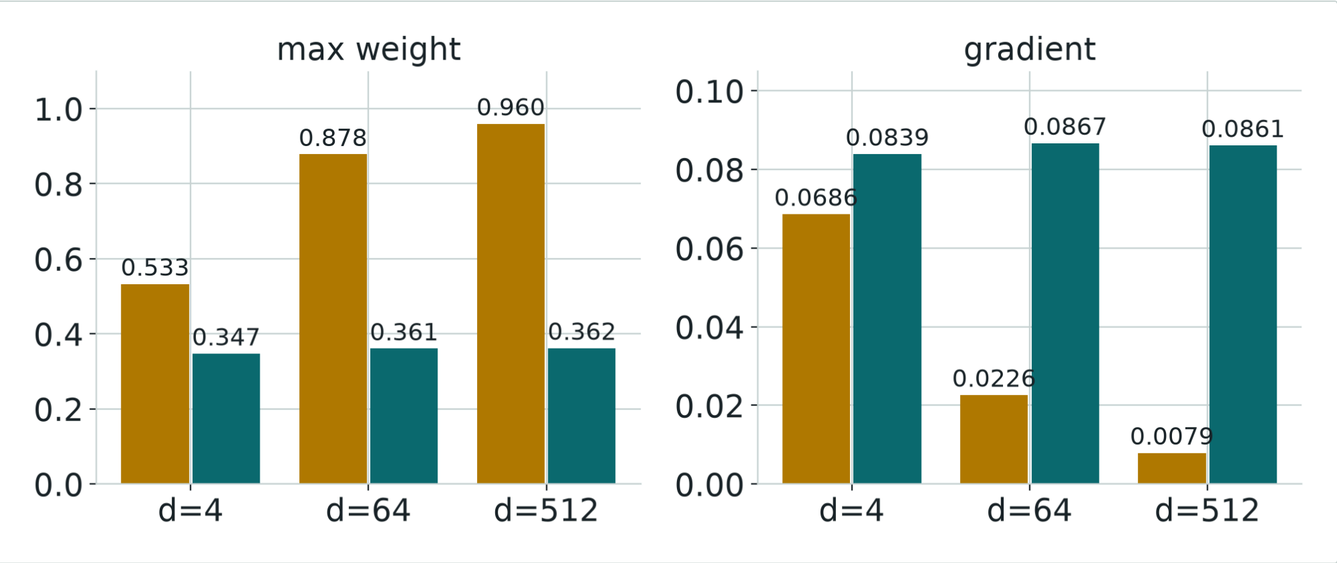
쿼리와 키는 **숫자 개수**가 같아야 한다 (내적)

키와 값은 **개수**가 같아야 한다 (섞기). 값의 숫자 개수는 자유

```
# 같은 계산을 einsum으로: 사라지는 글자(d, k)가 곱하고 더해 없어지는 차원
점수 = torch.einsum("qd,kd->qk", q, k) / math.sqrt(4)
print(torch.allclose(torch.einsum("qk,kv->qv", 점수.softmax(dim=-1), v), 출력))  # True
```

12.2 d가 커지면 무슨 일이 생기나

```
d=  4  점수 표준편차  2.00 → 1.00,  가장 큰 가중치 0.533 → 0.347,  기울기 0.0686 → 0.0839
d= 64  점수 표준편차  8.08 → 1.01,  가장 큰 가중치 0.878 → 0.361,  기울기 0.0226 → 0.0867
d=512  점수 표준편차 22.69 → 1.00,  가장 큰 가중치 0.960 → 0.362,  기울기 0.0079 → 0.0861
```



나누지 않음 · √d로 나눔 · 왼쪽 가장 큰 가중치, 오른쪽 기울기

- 쿼리 1000개, 쿼리마다 키 8개
- q, k는 평균 0, 표준편차 1인 난수
- d=512: 거의 언제나 **키 하나만** 보고, 기울기는 10분의 1도 안 된다

6차시 · 12.2 점수를 d의 제곱근으로 나누는 이유 · 예제 12-transformer/b03_scale.py (출력과 그림)

12.2 나누지 않으면 세 가지가 차례로 무너진다

1 점수가 커진다

내적은 숫자 d개를 곱해 **더한** 값. 표준편차가 √d배: 2, 8, 22.6

2 한 칸으로 쏠린다

12.1의 20배 실험과 같은 일. d=512면 가장 큰 가중치 0.960

3 기울기가 사라진다

0과 1에 붙은 확률은 움직이지 않는다. 0.0079 대 0.0861

스케일드 닷 프로덕트 어텐션 (크기를 맞춘 내적 어텐션)

```
softmax(Q @ K.T / sqrt(d)) @ V
```

직접 실행 10분

1. b01_soft_dict.py를 실행하고, 질문을 [0.0, 1.0] ("달기만 했으면")으로 바꿔 다시 실행한다. 가장 큰 가중치는 어느 음식에? 섞어 꺼낸 값은?
2. 같은 파일에서 20배 가중치 줄의 결과는 어떻게 바뀌나? 딕셔너리처럼 꺼냈나?
3. b03_scale.py의 d 목록에 2048을 더해 실행한다. 나누지 않았을 때 가장 큰 가중치와 기울기는?

```
cd examples/vol2/12-transformer && uv run python b01_soft_dict.py
```

d=512인 어텐션에서 점수를 √d로 나누지 않으면 어떻게 될까요?

Q

가 가중치가 모든 키에 고르게 퍼진다

나 키 하나에 쏠리고 기울기가 거의 0이 된다

다 출력의 모양이 (질문 수, 키 수)로 바뀐다

라 줄마다 가중치의 합이 1보다 커진다

6차시 정리

트랜스포머는 차례로 넘겨받지 않고 **모든 글자가 모든 글자를 한 번에** 본다. 어디를 볼지는 배운다

어텐션 = 쿼리와 키의 내적으로 닮은 정도 → 소프트맥스로 비율 → 그 비율로 값을 섞기. 부드러운 딕셔너리

점수를 **√d로 나눠야** 쏠리지 않고 기울기가 흐른다: softmax(Q @ K.T / sqrt(d)) @ V

다음 차시: 문장이 자기 자신을 본다 · 셀프 어텐션과 마스크

7차시

07

셀프 어텐션과 마스크

책 12.3 ~ 12.4절

학습 목표

- 같은 입력에서 쿼리·키·값을 만드는 셀프 어텐션을 nn.Module로 짠다
- 셀프 어텐션이 글자 순서를 모른다는 것을 실험으로 확인한다
- 패딩 마스크와 인과 마스크를 소프트맥스 전에 -inf로 적용한다

12.3 문장이 자기 자신을 본다

"그것" 문제에서 글자 하나는 **세 역할**을 동시에 해야 한다

쿼리로서

"나와 관련된 글자는 어디 있지?"

키로서

"나는 이런 글자야"

값으로서

"나를 보면 이 정보를 줄게"

역할마다 다른 모습이 필요하니 글자 벡터에 **서로 다른 nn.Linear 세 개**를 곱한다. 어떤 모습으로 바꿀지는 학습이 정한다

12.3 셀프 어텐션 층 만들기

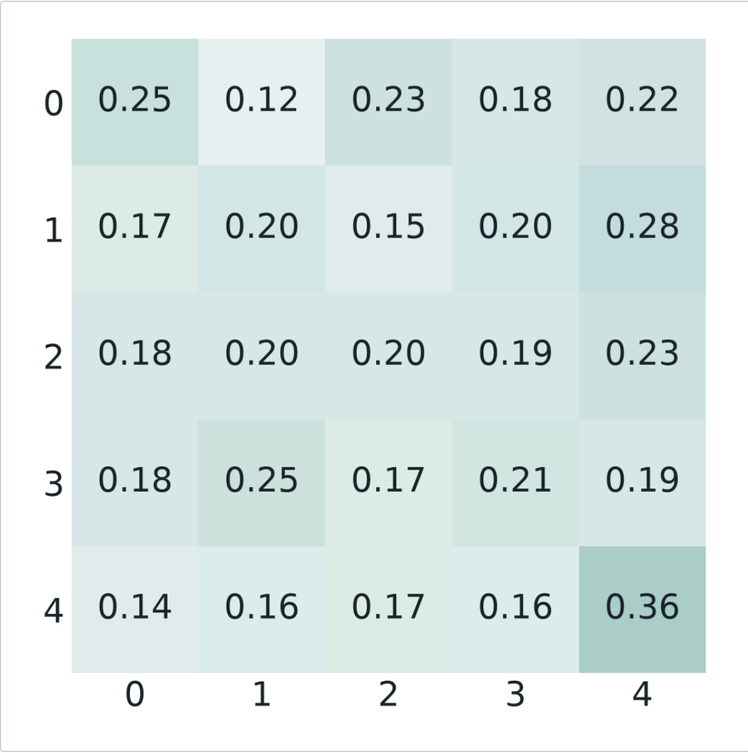
```
class 셀프_어텐션(nn.Module):
    def __init__(self, 차원: int) -> None:
        super().__init__()
        self.쿼리 = nn.Linear(차원, 차원, bias=False)   # "나는 무엇을 찾나"
        self.키 = nn.Linear(차원, 차원, bias=False)     # "나는 이런 정보를 가졌다"
        self.값 = nn.Linear(차원, 차원, bias=False)     # "나를 고르면 이걸 준다"

    def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
        q, k, v = self.쿼리(x), self.키(x), self.값(x)               # 모두 (B, T, C)
        점수 = q @ k.transpose(-2, -1) / math.sqrt(x.size(-1))      # (B, T, T)
        가중치 = 점수.softmax(dim=-1)
        return 가중치 @ v, 가중치
x = torch.randn(1, 5, 8)            # 문장 1개 × 글자 5개 × 글자마다 숫자 8개
출력, 가중치 = 셀프_어텐션(8)(x)
```

앞으로의 텐서 모양: **(배치 B, 글자 수 T, 숫자 개수 C)** · 자리마다 이름을 붙여 읽는다

7차시 · 12.3 같은 입력에서 쿼리·키·값 만들기 · 예제 12-transformer/b04_self_attention.py (핵심 부분)

12.3 글자 다섯 개가 글자 다섯 개를 보는 표



줄 = 지금 글자(쿼리) · 열 = 보는 글자(키)

```
입력 (1, 5, 8) → 출력 (1, 5, 8) / 가중치 (1, 5, 5)
파라미터 수: 192
```

- 출력 모양이 입력과 같다 → **겹쳐 쌓을 수 있다**
- 0번 줄: 자기 25%, 1번 12%, … 섞어 새 벡터
- 학습 전이라 비율이 고만고만
- 8 × 8 × 3 = 192개 (편향 없음)

12.3 어텐션은 순서를 모릅니다

```
x = torch.randn(1, 5, 차원)              # 글자 5개
순서 = torch.tensor([4, 2, 0, 3, 1])     # 글자 순서를 뒤섞는다
with torch.no_grad():
    원래 = 셀프_어텐션(x)
    섞은 = 셀프_어텐션(x[:, 순서])
print("섞은 입력의 출력 == 원래 출력을 같은 순서로 섞은 것:",
      torch.allclose(섞은, 원래[:, 순서], atol=1e-6))
```

```
섞은 입력의 출력 == 원래 출력을 같은 순서로 섞은 것: True
0번 글자의 출력, 원래    : tensor([ 0.0907, -0.0069,  0.1461, -0.1893])
0번 글자의 출력, 섞은 뒤 : tensor([ 0.0907, -0.0069,  0.1461, -0.1893])
```

글자들을 **주머니에 넣고 흔들어도** 결과는 같다: "고양이가 쥐를 쫓는다" = "쥐가 고양이를 쫓는다"

12.4 패딩 칸 가리기: 소프트맥스 전에 -inf

```
# 7장의 패딩 배치: 진짜 글자 3개인 문장과 5개인 문장. 0번은 채운 칸
입력 = torch.tensor([[13, 22, 19,  0,  0],
                     [ 7, 14,  1, 25, 21]])
진짜 = 입력 != 0                                          # (2, 5) 진짜 글자 자리
x = torch.randn(2, 5, 4)
점수 = x @ x.transpose(-2, -1) / math.sqrt(4)      # (2, 5, 5)

# 키 쪽(열)이 채운 칸이면 -inf로: softmax에서 e^(-inf) = 0이 된다
가린_점수 = 점수.masked_fill(~진짜[:, None, :], float("-inf"))
가중치 = 가린_점수.softmax(dim=-1)

# 잘못된 방법: softmax를 먼저 하고 채운 칸을 0으로 지우면 합이 1이 아니다
틀린 = 점수.softmax(dim=-1) * 진짜[:, None, :]
```

~진짜[:, None, :] 는 (2, 1, 5): 브로드캐스팅으로 **모든 줄에 같은 열 마스크** · "어떤 글자가 묻든, 채운 칸(열)은 보지 마"

12.4 채운 칸 열은 0, 줄마다 합은 1



첫 문장 · 회색 = 가린 칸(채운 칸 3, 4번 열)

바른 방법: softmax 전에 -inf

줄마다 합: `tensor([1.000, 1.000, 1.000, 1.000, 1.000])`

흔한 실수: softmax 뒤에 0으로 지우기

softmax 뒤에 지우면 줄마다 합:
`tensor([0.567, 0.910, 0.573, 0.374, 0.303])`

진짜 글자 세 개만으로 비율을 **다시 나눠** 가졌다

12.4 앞으로 올 글자 가리기: 인과 마스크

```
T = 5
보임 = torch.tril(torch.ones(T, T, dtype=torch.bool))
       # 아래 삼각형: 나와 내 앞만 보임

def 인과_어텐션(x):
    점수 = x @ x.transpose(-2, -1) / math.sqrt(x.size(-1))
    가중치 = 점수.masked_fill(~보임, float("-inf")).softmax(dim=-1)
    return 가중치 @ x, 가중치

# 마지막 글자를 바꿔도 앞 글자들의 출력은 그대로일까?
바꾼_x = x.clone()
바꾼_x[0, -1] = 100.0
```

```
print(보임.int())

[[1, 0, 0, 0, 0],
 [1, 1, 0, 0, 0],
 [1, 1, 1, 0, 0],
 [1, 1, 1, 1, 0],
 [1, 1, 1, 1, 1]]
```

학습 때는 문장 전체를 한 번에 넣으니 뒤 글자를 훔쳐볼 수 있다: **시험 보면서 답안지 보기**. 그래서 자기와 자기 앞만 보게 가린다

7차시 · 12.4 인과 마스크 · 예제 12-transformer/b07_causal_mask.py (핵심 부분, 출력은 tensor(...)와 dtype를 줄여 적음)

12.4 뒤 글자를 바꿔도 앞은 그대로



회색 = 가린 칸(앞으로 올 글자)

```
앞 네 글자 출력이 같은가: True
마지막 글자 출력이 같은가: False
```

앞 글자들은 마지막 글자를 **아예 볼 수 없다**

그래서 문장 하나로 "첫 글자 보고 둘째 맞히기", "두 글자 보고 셋째 맞히기", … 를 **한 번의 계산으로 동시에** 학습한다. RNN처럼 차례를 기다리지 않는다

7차시 · 12.4 인과 마스크 · 예제 12-transformer/b07_causal_mask.py (출력, 가중치를 열지도로)

12.4 모든 칸이 가려진 줄이 생기면?

`e^(-inf) / (e^(-inf) + … + e^(-inf)) = 0 / 0 = nan`

그 줄 전체가 nan이 되고, 다음 층으로 퍼지면 **학습 전체가 망가진다**

패딩 마스크

열(키) 쪽에만 적용한다. 줄(쿼리)까지 가리면 nan. 채운 칸 줄의 출력은 손실에서 뺀다

인과 마스크

대각선(자기 자신)이 언제나 보이니 이런 일이 생기지 않는다

직접 실행 10분

1. b07_causal_mask.py에서 바꾸는 글자를 가운데로 (바꾼_x[0, 2] = 100.0) 옮긴다. 어느 글자의 출력이 그대로일지 **먼저 적고**, 아래 줄을 더해 확인한다
2. b05_order_blind.py의 셀프_어텐션 함수에 인과 마스크를 넣고 섞기 실험을 다시 한다. 결과는 True일까 False일까?

```
for i in range(5):                                          # 1번 과제: b07 끝에 더한다
    print(f"{i}번 글자 출력이 같은가:", torch.allclose(출력[:, i], 바꾼_출력[:, i]))

보임 = torch.tril(torch.ones(5, 5, dtype=torch.bool))  # 2번 과제: b05에
점수 = 점수.masked_fill(~보임, float("-inf"))        # softmax 바로 앞에
```

패딩 칸을 바르게 가리는 방법은?

Q

가 softmax 뒤에 채운 칸 열을 0으로 곱한다

나 softmax 전에 채운 칸 열(키)의 점수를 -inf로

다 softmax 전에 채운 칸의 줄과 열을 모두 -inf로

라 채운 칸의 입력 벡터를 0으로 바꿔 둔다

7차시 정리

셀프 어텐션은 같은 입력에서 nn.Linear 세 개로 쿼리·키·값을 만든다. 출력 모양이 입력과 같아 쌓을 수 있다

마스크가 없으면 어텐션은 **순서를 모른다**: 섞어 넣으면 출력도 그대로 섞일 뿐

마스크는 소프트맥스 전에 -inf. 패딩은 열(키) 쪽에만, 인과 마스크는 tril로 나와 내 앞만

다음 차시: 여러 관점으로 동시에 · 멀티헤드, 위치 정보, 블록 조립

8차시

08

멀티헤드, 위치 정보, 블록 조립

책 12.5 ~ 12.7절

학습 목표

- 헤드를 나누고 합치는 모양 바꾸기를 따라가고, 내장 부품과 결과를 맞춘다
- 위치 정보를 더하는 두 가지 방법(사인·코사인, 학습)을 안다
- 층 정규화·잔차·피드포워드로 블록을 조립하고 $12C^2$로 센다

12.5 여러 관점으로 동시에 보기

"고양이가 생선을 **먹지** 않았다"에서 "먹지"가 봐야 할 곳: **누가**(고양이), **무엇을**(생선), **바로 앞 글자**

어텐션 한 번

비율 하나로 섞는다. 여러 곳을 **서로 다른 이유로** 보기 어렵다

멀티헤드 어텐션

숫자 C개를 H조각(**헤드**)으로 나눠 조각마다 따로 어텐션. 여러 사람이 각자 관점으로 읽고 의견을 모은다

`C = 8` → `헤드 0: 숫자 0~3` `헤드 1: 숫자 4~7` H = 2, D = C ÷ H = 4

12.5 헤드를 나누고 합치기

```
B, T, C, H = 2, 5, 8, 2                 # 문장 2개, 글자 5개, 숫자 8개, 헤드 2개
D = C // H                              # 헤드마다 숫자 4개씩
x = torch.arange(B * T * C, dtype=torch.float32).view(B, T, C)
나눔 = x.view(B, T, H, D).transpose(1, 2)        # (B, T, C) → (B, T, H, D) → (B, H, T, D)
결과 = 나눔 @ torch.eye(D)                         # 행렬 곱의 결과는 (B, H, T, D) 순서로 새로 놓인다
되돌림 = 결과.transpose(1, 2)                      # (B, H, T, D) → (B, T, H, D)
try:
    되돌림.view(B, T, C)
except RuntimeError as e:
    print("view 오류:", str(e).split("(")[0].strip())
합침 = 되돌림.contiguous().view(B, T, C)           # 4장의 permute → contiguous → view
```

```
나눈 모양: (2, 2, 5, 4) 이어져 있나? False
0번 헤드 = 숫자 0~3번 칸: True
1번 헤드 = 숫자 4~7번 칸: True
view 오류: view size is not compatible with input tensor's size and stride
다시 합친 모양: (2, 5, 8) 원래와 같은가? True
```

8차시 · 12.5 헤드를 나누고 합치기 · 예제 12-transformer/b08_split_heads.py (print 줄 일부 생략)

12.5 멀티헤드 어텐션 직접 만들기

```
class 멀티헤드_어텐션(nn.Module):
    def __init__(self, 차원: int, 헤드_수: int) -> None:
        super().__init__()
        self.헤드_수 = 헤드_수
        self.qkv = nn.Linear(차원, 3 * 차원)        # 쿼리·키·값을 한 번에 만든다
        self.출력 = nn.Linear(차원, 차원)            # 헤드들의 결과를 섞는다
    def forward(self, x: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)                    # 각각 (B, T, C)
        q, k, v = (t.view(B, T, self.헤드_수, -1).transpose(1, 2) for t in (q, k, v))
        점수 = torch.einsum("bhqd,bhkd->bhqk", q, k) / math.sqrt(k.size(-1))   # (B, H, T, T)
        미래 = torch.ones(T, T, dtype=torch.bool).triu(1)          # 대각선 위 = 앞으로 올 글자
        가중치 = 점수.masked_fill(미래, float("-inf")).softmax(dim=-1)
        y = (가중치 @ v).transpose(1, 2).contiguous().view(B, T, C)
        return self.출력(y), 가중치
```

```
출력 (2, 6, 16) 가중치 (2, 4, 6, 6)
파라미터 수: 1088
```

816 (qkv) + 272 (출력). 헤드를 나눠도 **파라미터 수는 그대로**

8차시 · 12.5 멀티헤드 어텐션 직접 만들기 · 예제 12-transformer/b09_mha.py (핵심 부분, x = torch.randn(2, 6, 16), 헤드_수=4)

12.5 내장 부품과 결과 맞춰 보기

```
파이토치 = nn.MultiheadAttention(16, num_heads=4, batch_first=True)
파이토치.in_proj_weight.copy_(우리.qkv.weight)      # (3C, C): 쿼리·키·값 순서로 쌓여 있다
…                                                  # 편향과 출력 층도 똑같이 복사
미래 = torch.ones(6, 6, dtype=torch.bool).triu(1)        # True = 보지 마
그쪽_출력, 그쪽_가중치 = 파이토치(x, x, x, attn_mask=미래, average_attn_weights=False)
```

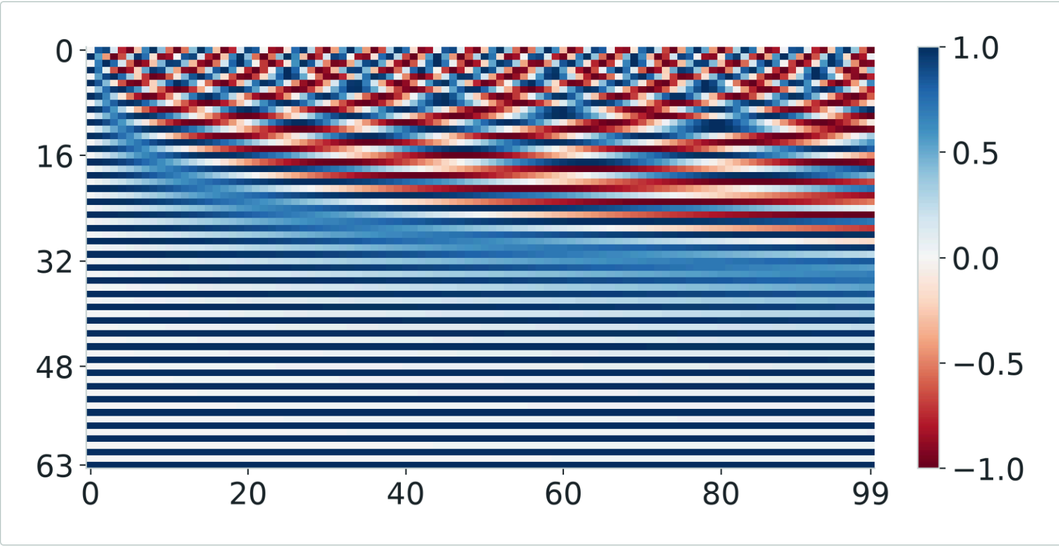
```
한줄 = F.scaled_dot_product_attention(q, k, v, is_causal=True)    # PyTorch가 준비해 둔 한 줄짜리 함수
한줄2 = F.scaled_dot_product_attention(q, k, v, attn_mask=보임)   # True인 칸만 본다
```

```
파라미터 수: 1088
출력이 같은가: True
가중치가 같은가: True
모양: (2, 4, 6, 8) 같은가: True
패딩 마스크도 같은가: True
```

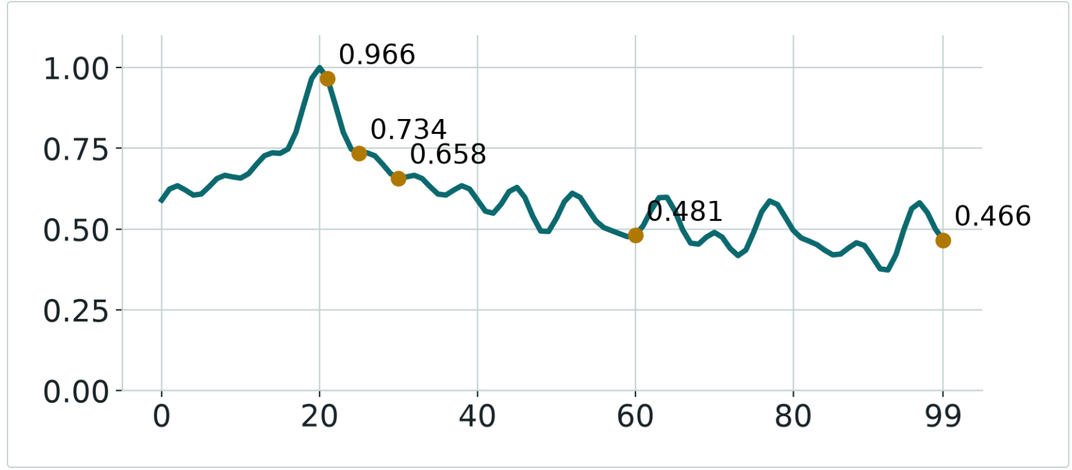
마스크의 True	뜻
nn.MultiheadAttention	**가린다**
F.scaled_dot_product_attention	**본다**

12.6 순서 알려 주기: 사인·코사인 위치 인코딩

초침은 빨리, 분침은 천천히, 시침은 더 천천히: 바늘들을 함께 보면 **몇 번째 자리인지** 안다. 그 값을 글자 벡터에 **더한다**



가로 위치(몇 번째 글자) · 세로 칸 번호 0~63 · 위쪽 칸 = 빠른 바늘



위치 20과의 코사인 유사도 · **가까운 위치끼리 더 닮았다**

12.6 학습하는 위치 임베딩 (GPT의 방식)

```
글자_임베딩 = nn.Embedding(10, 4)            # 글자 10종류, 글자마다 숫자 4개 (학습되는 표)
번호들 = torch.tensor([[3, 7, 3]])           # 글자 번호 세 개 (첫째와 셋째가 같은 글자)
x = 글자_임베딩(번호들)

# 위치 임베딩을 더하면 같은 글자라도 자리에 따라 다른 숫자가 된다
위치_임베딩 = nn.Embedding(8, 4)             # 위치 0~7, 위치마다 숫자 4개
x = x + 위치_임베딩(torch.arange(3))
```

```
표의 3번 줄을 꺼낸 것과 같은가: True
위치를 더한 뒤 0번 == 2번 자리: False
위치가 있으면 섞은 결과가 원래를 섞은 것과 같은가: False
```

사인·코사인: 공식이라 어떤 길이든 계산된다

학습: 단순하고 데이터에 맞춰지지만 표에 없는 길이는 못 다룬다

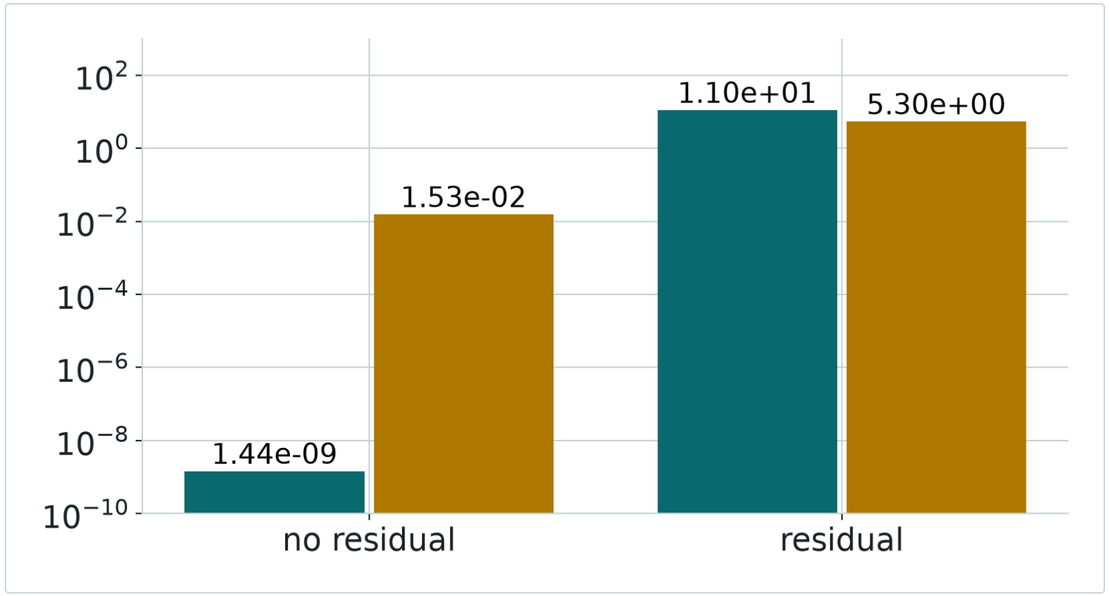
12.7 층 정규화와 잔차 연결

층 정규화: 글자마다 따로 표준화

```
층정규화 = nn.LayerNorm(4)
# [1, 2, 3, 4]와 [10, 20, 30, 40] →
[-1.342, -0.447,  0.447,  1.342]
[-1.342, -0.447,  0.447,  1.342]
글자 1개만 넣어도 된다: (1, 1, 4)
다른 글자와 섞이지 않는다: True
```

자기 숫자들로 **dim=-1** 방향 평균·분산. 배치 정규화(dim=0)와 달리 패딩·배치 크기와 무관, train/eval 동작이 같다

잔차: h + 층(h), 32층 tanh



■ 첫 층 기울기 · ■ 마지막 층 기울기 · 로그 눈금

12.7 블록 하나 완성하기

```
self.피드포워드 = nn.Sequential(nn.Linear(차원, 4 * 차원), nn.GELU(), nn.Linear(4 * 차원, 차원))

def forward(self, x: torch.Tensor) -> torch.Tensor:
    x = x + self.어텐션(self.정규화1(x))        # 글자끼리 정보 주고받기
    x = x + self.피드포워드(self.정규화2(x))    # 글자마다 혼자 생각하기
    return x
```

```
입력 (2, 10, 128) → 출력 (2, 10, 128)
  정규화1: 256개
  어텐션: 66,048개
  정규화2: 256개
  피드포워드: 131,712개
합계: 198,272개 (12 × C² = 196,608)
nn.TransformerEncoderLayer: 198,272개
```

회의(어텐션) 뒤 각자 자리에서 **정리**(피드포워드)

정규화를 **앞에**: Pre-LN, GPT-2 이후 대부분

$4C^2 + 8C^2 =$ **블록 하나에 $12C^2$**

12.7 인코더, 디코더, 그리고 교차 어텐션

```
교차_어텐션 = nn.MultiheadAttention(16, num_heads=4, batch_first=True)
원문 = torch.randn(1, 4, 16)          # 인코더가 읽은 원문 단어 4개 (예: 영어 문장)
번역문 = torch.randn(1, 7, 16)        # 디코더가 지금까지 쓴 번역문 7글자
출력, 가중치 = 교차_어텐션(번역문, 원문, 원문)   # 쿼리는 번역문, 키·값은 원문
```

```
출력 (1, 7, 16) 가중치 (1, 7, 4)
번역문 0번 글자가 원문 단어들을 보는 비율: tensor([0.25, 0.36, 0.18, 0.21])
```

종류	마스크	대표 모델	잘하는 일
인코더만	없음 (앞뒤 다 봄)	BERT	문장 이해, 분류 (13장)
디코더만	인과 마스크	**GPT**	글 이어 쓰기, 대화 (14장)
인코더-디코더	디코더에만 인과	원래 트랜스포머, T5	번역, 요약

직접 실행 10분

1. b09_mha.py에서 헤드_수를 1, 2, 8로 바꿔 (nn.MultiheadAttention의 num_heads도 같이) 실행한다. 가중치 모양과 파라미터 수는? 내장 부품과 여전히 같은가?
2. b15_block.py에서 C = 256으로 바꿔 실행한다. 블록 하나의 파라미터 합계는 $12 \times C^2$에 얼마나 가까운가?
3. 예측해 보기: 9차시의 작은 GPT(C = 128, 블록 4개)에서 블록들의 파라미터는 모두 몇 개일까?

```
cd examples/vol2/12-transformer && uv run python b15_block.py
```

transpose(1, 2) 뒤에 바로 view(B, T, C)를 하면 오류가 나는 이유는?

Q

가 원소 개수가 달라져서

나 메모리가 보이는 순서대로 이어져 있지 않아서

다 헤드 수가 C를 나누지 못해서

라 view는 4차원 텐서를 받지 않아서

8차시 정리

멀티헤드: view(B, T, H, D).transpose(1, 2)로 나누고 transpose(1, 2).contiguous().view(B, T, C)로 합친다. 파라미터 수는 그대로

위치 정보: 사인·코사인 또는 학습하는 위치 임베딩을 글자 벡터에 더한다

블록: x + 어텐션(층정규화(x)), x + 피드포워드(층정규화(x)). 하나에 약 $12C^2$

다음 차시: 부품을 모아 헌법과 민법을 읽는 작은 GPT

9차시

09

작은 GPT 학습시키고 들여다보기

책 12.8 ~ 12.9절

학습 목표

- 글을 "다음 글자 맞히기" 문제로 바꾸고 첫 손실로 모델을 점검한다
- 학습 곡선에서 과적합을 읽고, 학습한 모델로 글을 만든다
- 어텐션이 어디를 보는지 들여다보고 문맥 길이의 대가를 계산한다

12.8 학습 재료: 헌법과 민법

대한민국헌법(1987년 전부개정)과 민법(법률 제17503호) 조문 · 저작권 보호를 받지 않는 저작물(저작권법 제7조) · 위키문헌에서 받는다

```
글자 165,422개, 줄 2,454개
  제1장 총강
  제1조 ① 대한민국은 민주공화국이다.
  ② 대한민국의 주권은 국민에게 있고, 모든 권력은 국민으로부터 나온다.
글자 종류: 478
[348, 9, 350, 1, 20, 1, 114, 439, 196, 63, 321]
제1조 ① 대한민국은
```

글자	횟수
띄어쓰기	34,984
의	5,817
에	3,931
한	3,481
는	3,357
제	3,192

글자표: 글자마다 번호 붙이기 (7장 7.7절) · **인코딩**은 글 → 번호 텐서, **디코딩**은 번호 → 글

12.8 다음 글자 맞히기 문제 만들기

```
학습 148,879글자, 검증 16,543글자
입력 모양: (2, 12) 정답 모양: (2, 12)
입력 0: '대법원과 각급법원으로 '
정답 0: '법원과 각급법원으로 조'
입력 1: '재로 주채무를 소멸하게'
정답 1: '로 주채무를 소멸하게 '
  '대' 다음은 → '법'
  '대법' 다음은 → '원'
  '대법원' 다음은 → '과'
  '대법원과' 다음은 → ' '
  '대법원과 ' 다음은 → '각'
```

정답 = 입력을 **한 글자 뒤로 민 것**. 글 자체가 문제이자 정답

조각 하나에 문제가 문맥 길이만큼. 인과 마스크 덕분에 **한 번에** 풀고 채점: 128 × 32 = **4,096문제**

앞 90% 학습, 뒤 10% 검증. 섞지 않고 **앞뒤로 자른다**

12.8 작은GPT.py: 부품을 모으면




```
@dataclass
class 설정:
    어휘_크기: int
    문맥_길이: int = 128        # 한 번에 보는 글자 수
    차원: int = 128              # 글자 하나를 나타내는 숫자 개수
    헤드_수: int = 4
    층_수: int = 4
    드롭아웃: float = 0.1
    위치_사용: bool = True      # 실습용 스위치: 위치 임베딩을 더할까
    인과_마스크: bool = True    # 실습용 스위치: 앞으로 올 글자를 가릴까
```

학습 함수 = 6장의 다섯 줄 루프 + 8장 도구: **AdamW**(weight decay 0.1) · **코사인 스케줄러** · **그래디언트 클리핑** 1.0

12.8 학습 전 점검: 크기와 첫 손실

```
  글자_임베딩: 61,184개
  위치_임베딩: 16,384개
  블록들: 793,088개
  마지막_정규화: 256개
  머리: 61,662개
모두 932,574개
입력 (32, 128) → 로짓 (32, 128, 478)
```

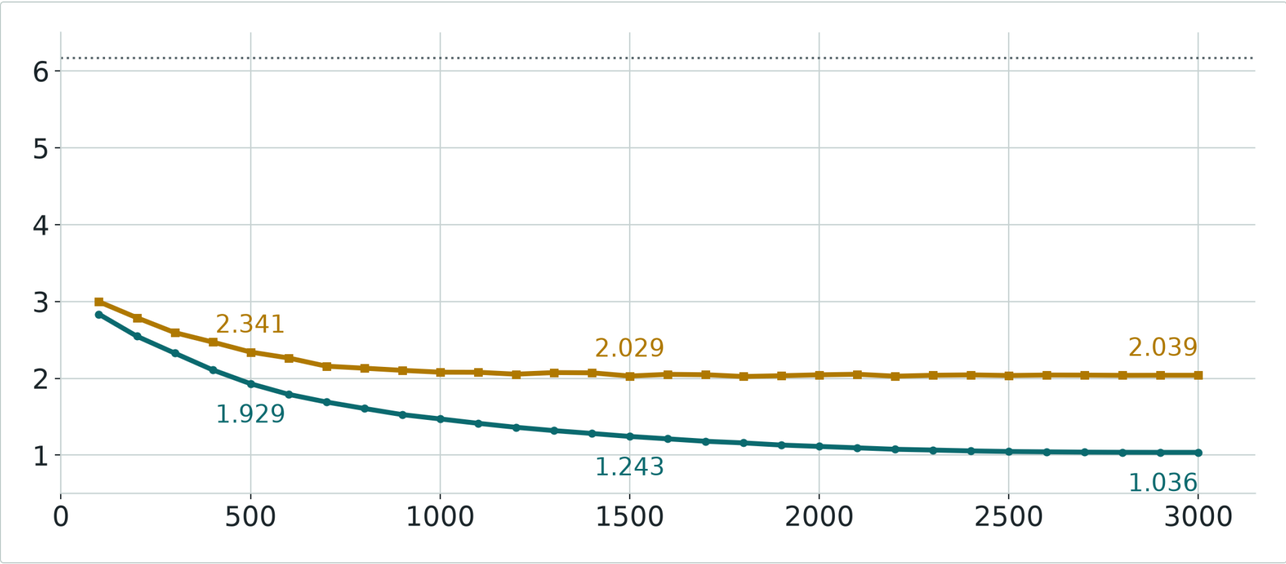
블록들 = 198,272 × 4 · float32로 약 **3.6MB**

70억 개짜리 모델은 26GB였다

`학습 전 손실 6.3229, 찍기 수준 -ln(1/478) = 6.1696`

478종류를 고르게 찍는 수준과 거의 같다: 아직 아무것도 모르고, 엉뚱한 글자를 강하게 확신하지도 않는 **건강한 출발**

12.8 3,000단계 학습시키기



● 학습 손실 · ■ 검증 손실 · 점선 찍기 수준 6.17 · 가로 단계

```
장치: mps
  500단계  학습 손실 1.929  검증 손실 2.341
 1000단계  학습 손실 1.471  검증 손실 2.079
 1500단계  학습 손실 1.243  검증 손실 2.029
 2000단계  학습 손실 1.113  검증 손실 2.045
 2500단계  학습 손실 1.047  검증 손실 2.036
 3000단계  학습 손실 1.036  검증 손실 2.039
```

1,500단계 뒤 검증은 제자리, 차이 1 가까이: **과적합**

e^(-2.03) ≈ 0.13: 못 본 조문에서도 정답에 평균 13% 안팎

12.9 한 글자씩 이어 쓰기

시작 글 → **마지막 자리**의 예측에서 확률대로 한 글자 뽑기(torch.multinomial) → 뒤에 붙여 다시 넣기 · 100글자 · 최근 128자만 본다

제850조(혼인신고의 방법, 성년후견인의 정이해관) 제83조제3호, 제851조 및 제925조의 규정에 위반하거나 부양을 할 수 있다. 이 경우 가정법원은 본인, 친족, 그 취소 및 청구, 제

국민은 안전보장에 위보장된다.
제13조 ① 한정안은 주각부의 대법원·비속에 관하여 관한 조약되음에 관하여 법률로 정한다.
② 제1항 제2항의 규정은 미성년자의 직권으로 지정한다.
제41

```
학습 데이터에 있나? False  ← '제83조제3호, 제851조 및 제925조의 규정에 위반하거나'
학습 데이터에 있나? False  ← '혼인신고의 방법'
```

형식(조 번호, 괄호 속 제목, ①② 항, "법률로 정한다")은 지키고 **뜻**은 엉망. 외운 문장이 아니라 형식을 새로 조립했다

9차시 · 12.9 한 글자씩 이어 쓰기 · 예제 12-transformer/b21_generate.py (세 시작 글 중 둘, 1초 안팎)

12.9 어텐션은 어디를 보고 있을까

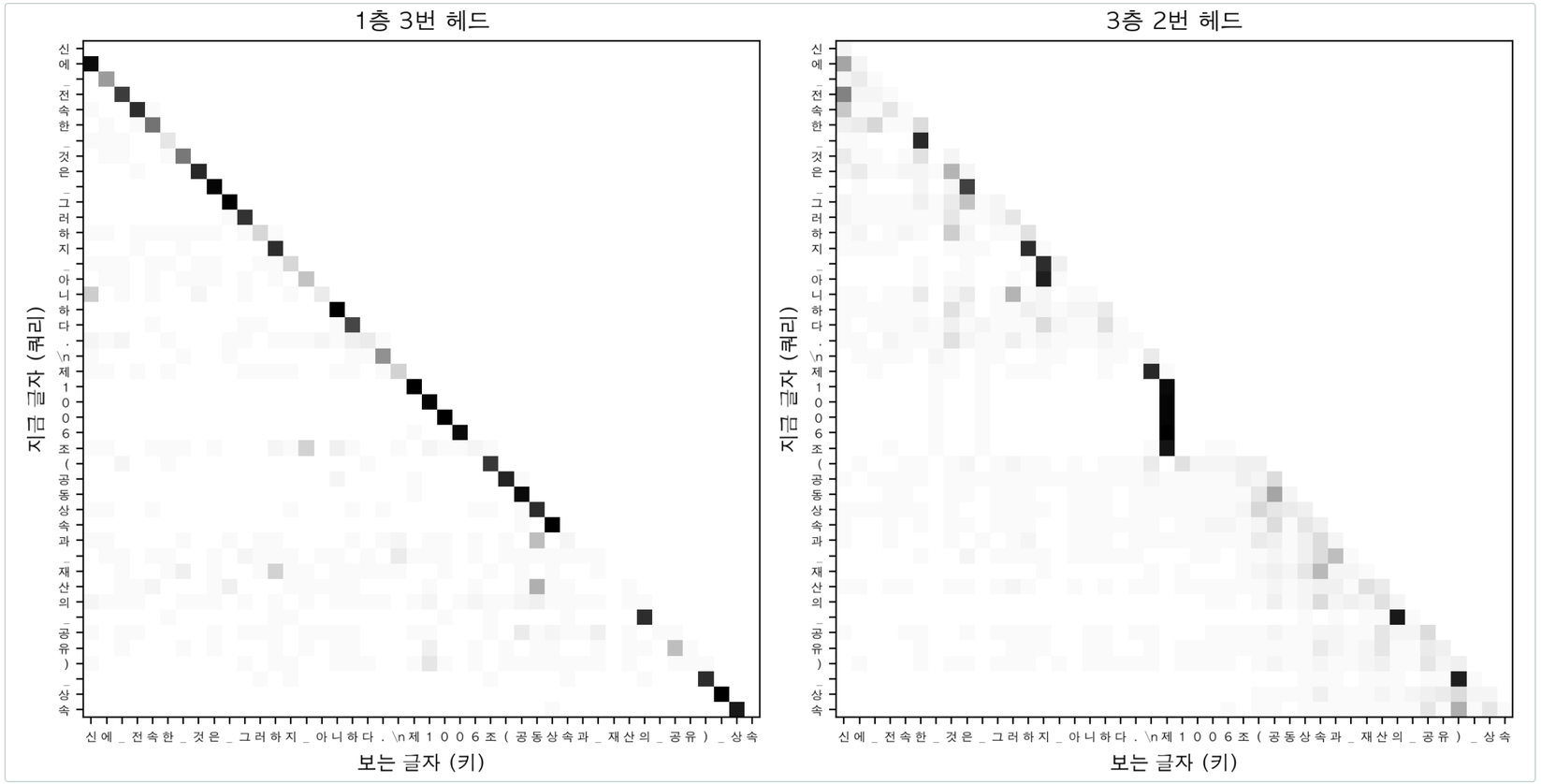
1층 헤드	평균 몇 칸 앞	바로 앞 글자에
0번	34.9칸	0.02
1번	30.1칸	0.05
2번	8.5칸	**0.74**
3번	9.4칸	**0.71**

아무도 가르치지 않았는데 두 헤드는 **바로 앞 글자**를, 두 헤드는 **넓게** 본다. 헤드마다 일이 나뉘었다

```
'속인의 일신에 전속한 것은 그러하지 아니하다.\n제1006' 다음 글자 예측: [('조', 0.999), ('(', 0.0)]
1층: ' 승'['계'] 0.03 | '에 '['전'] 0.06 | '10'['0'] 0.80 | '10'['0'] 0.96
2층: '다.'['\n'] 0.41 | ' 아'['니'] 0.10 | '10'['0'] 0.16 | '다.'['\n'] 0.19
3층: '10'['0'] 0.36 | '10'['0'] 0.17 | '.\n'['제'] 0.98 | '10'['0'] 0.29
4층: '10'['0'] 0.34 | '다.'['\n'] 0.60 | '제1'['0'] 0.51 | '\n제'['1'] 0.33
```

"제1006" 다음은 "조"를 0.999로 확신. 그 순간 3층 2번 헤드는 조문 번호가 시작된 **"제"**를 0.98로 보고 있다

12.9 학습된 헤드의 무늬



왼쪽 1층 3번: **대각선** 바로 아래 칸 = 바로 앞 글자 보기

오른쪽 3층 2번: "제1006조"가 모두 "제"를 보는 **세로 줄**

검을수록 많이 본 곳. 가중치는 어디서 가져왔는가일 뿐, 무엇을 했는지는 말해 주지 않는다

9차시 · 12.9 어텐션 들여다보기 · 예제 12-transformer/b22_look.py가 저장한 attention_heads.png (책과 같은 그림)

12.9 문맥 길이의 대가: 제곱으로 커진다



가로 문맥 길이(글자) · 세로 GB, 로그 눈금 · 헤드 32, bfloat16, 층 하나

```
바이트 = 헤드_수 * T * T * 칸_크기
```

가중치 표는 (헤드, T, T). 글자 8배 → 표 **64배** (0.062GB → 4GB)

128K 글자면 층 하나에 **1TB**: 어떤 GPU에도 안 올라간다

그래서 FlashAttention(표를 통째로 만들지 않기), KV 캐시

직접 실행 10분

1. b21_generate.py의 시작 글을 ["제1조", "혼인은"]으로 바꿔 실행한다. 어떤 형식을 흉내 내나? 뜻이 통하는 문장이 있나?
2. b23_memory.py의 문맥 목록에 32,768을 더해 실행한다. 가중치 표는 몇 GB인가? 8,192글자의 몇 배인가?
3. b19_first_loss.py의 "블록들" 숫자가 8차시 직접 실행에서 예측한 값과 같은지 확인한다

```
cd examples/vol2/12-transformer && uv run python b21_generate.py
```

9차시 · 직접 실행 · data/작은GPT.pt가 있으면 세 파일 모두 1초 안팎 (없으면 b21이 학습부터 해서 수십 초)

어휘가 478종류일 때 학습 전 손실 6.32가 "건강한 출발"인 이유는?

가 -ln(1/478) = 6.17, 고르게 찍는 수준과 거의 같아서

나 손실이 클수록 학습할 거리가 많아서

다 파라미터 수(932,574)에 비해 작아서

라 검증 손실보다 낮아서

9차시 정리

다음 글자 맞히기: 한 글자 밀린 조각이 정답. 인과 마스크 덕분에 조각 하나로 문제 여러 개를 동시에

첫 손실은 **-ln(1/어휘 크기)** 근처. 학습과 검증 손실을 나란히 보면 과적합이 보인다

만들 때는 마지막 자리의 확률대로 한 글자씩. 헤드는 저절로 일을 나누고, 문맥 비용은 **제곱**으로 큰다

다음 차시: 부품을 하나씩 빼 보는 실습과 12장 정리

10차시

10

실습: 부품을 하나씩 빼 보기

책 12.10 ~ 12.11절

학습 목표

- 위치 임베딩과 인과 마스크를 뺀 모델을 학습시켜 기본 모델과 비교한다
- 손실이 너무 좋으면 정답이 새는지 먼저 의심하는 습관을 들인다
- 12장의 부품을 한 장씩 다시 꿰어 정리한다

12.10 스위치만 바꿔 세 모델 학습하기

```
실험들 = {
    "기본": 설정(어휘_크기=len(표)),
    "위치 임베딩 없음": 설정(어휘_크기=len(표), 위치_사용=False),
    "인과 마스크 없음": 설정(어휘_크기=len(표), 인과_마스크=False),
}
for 이름, s in 실험들.items():
    print(f"[{이름}]")
    모델 = 학습(s, 학습_데이터, 검증_데이터, 단계_수=1000, 장치=장치, 보고_간격=1000)
    torch.manual_seed(0)
    글 = 표.디코딩(모델.생성(표.인코딩("제5조 ").unsqueeze(0), 개수=60)[0])
    print("  생성:", 글.replace("\n", " / "))
```

예상해 보기: 세 모델 중 손실이 가장 낮은 것은? 글을 가장 잘 쓰는 것은?

같은 시드, 같은 1,000단계: **스위치 하나만** 다르다

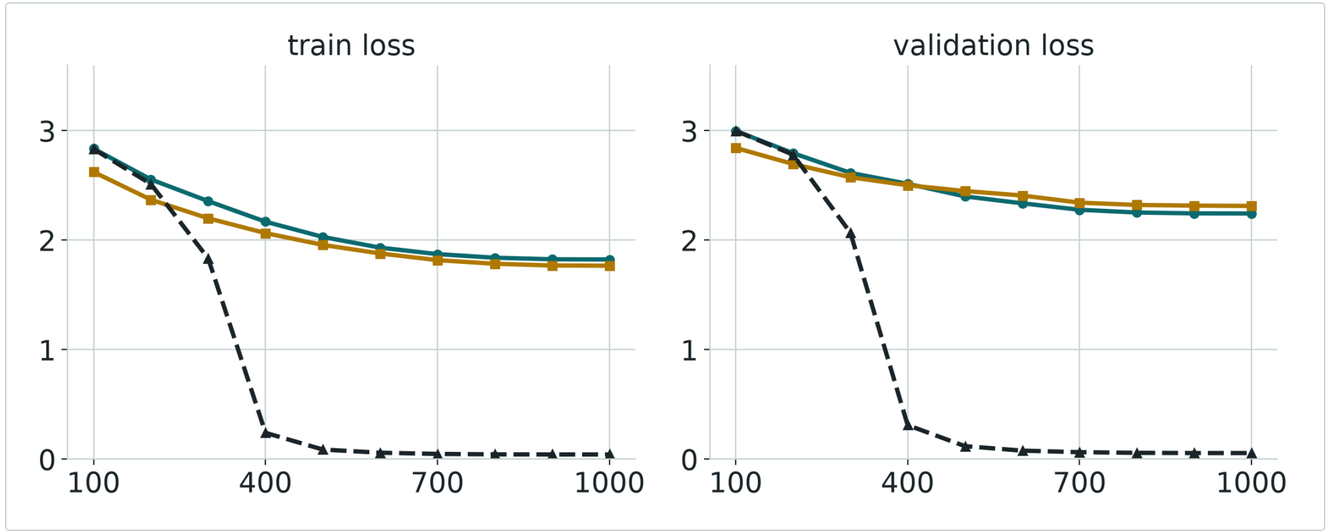
직접 실행 12분

1. b24_lab.py 실행을 **먼저 걸어 둔다** (MPS에서 36초, CPU는 몇 배)
2. 도는 동안 세 모델의 학습 손실·검증 손실 순위를 예상해 적는다
3. 결과가 나오면 표를 채우고, 예상과 다른 것에 동그라미를 친다
4. "인과 마스크 없음"이 만든 글을 소리 내어 읽어 보고, 그 손실과 비교한다

```
cd examples/vol2/12-transformer && uv run python b24_lab.py
```

12.10 결과: 가장 좋은 점수가 가장 나쁜 글

1,000단계	학습	검증
기본	1.824	2.244
위치 임베딩 없음	1.767	2.313
인과 마스크 없음	**0.043**	**0.056**



● 기본 · ■ 위치 임베딩 없음 · ▲ **점선 인과 마스크 없음** · 왼쪽 학습, 오른쪽 검증

```
[인과 마스크 없음]
  생성: 제5조 5조조55조조조555조조5조조5조조조조조조55조조조5조555555조조55조조조조조조5555조5조55조조55조
```

12.10 인과 마스크를 빼면: 정답을 베낀다

학습할 때

t번째 자리가 **t + 1번째 글자**, 즉 정답을 그대로 볼 수 있다. "다음 글자 맞히기" 대신 "바로 오른쪽 글자 베끼기"를 배운다. 검증도 똑같이 답을 보니 0.056

글을 만들 때

마지막 자리 오른쪽에는 **아무것도 없다**. 베낄 게 없으니 엉터리 글자만 나온다: "5조조55조조조…"

손실이 믿기 어려울 만큼 좋으면, 먼저 답이 새고 있지 않은지 의심하라

2장의 누수는 데이터를 나눌 때, 이것은 **모델 구조 안**의 누수. 오류 메시지도 없다. 7차시의 "뒤 글자를 바꿔도 앞 출력이 그대로인가" 테스트가 예방법

12.10 위치 임베딩을 빼면: 예상보다 멀쩡하다

1,000단계	학습 손실	검증 손실
기본	1.824	**2.244**
위치 임베딩 없음	**1.767** (더 낮다)	2.313 (더 높다)

비밀은 인과 마스크

0번 자리는 1글자, 100번 자리는 101글자를 섞는다. 몇 글자를 섞었는지에서 **대략 몇 번째인지** 짐작한다 (7차시 직접 실행 2번)

그래도 위치는 넣는다

"바로 앞 글자" 같은 가까운 자리 구분은 어렵다. 못 본 글에서는 **있는 쪽이 낫다**

12.10 더 해 볼 실험 (강사 시연)

1,000단계, 하나만 바꿈	학습 손실	검증 손실	MPS 시간
기본 (헤드 4, 드롭아웃 0.1)	1.824	2.244	b24 참고
헤드_수 = 1	1.769	**2.184**	13초
헤드_수 = 8	1.835	2.271	17초
드롭아웃 = 0.2	1.999	2.381	15초

1,000단계의 작은 모델에서는 **헤드 1개**가 가장 낫다. 많이 나눈다고 늘 좋지는 않다

드롭아웃 0.2: 학습·검증 차이 0.382 (기본 0.420)로 조금 줄었지만 둘 다 나빠졌다

숙제: 층_수 2·6, 문맥_길이 32·256, 법전.py의 원문들에 다른 법률 더하기

12.11 12장 정리 ①: 어텐션의 부품

부품	한 줄 요약
어텐션	쿼리·키 내적 → 소프트맥스 비율 → 값 섞기. softmax(Q @ K.T / sqrt(d)) @ V
√d로 나누기	내적이 √d배로 커져 쏠리고 기울기가 사라지는 것을 막는다
셀프 어텐션	같은 입력에서 nn.Linear로 q, k, v. 마스크가 없으면 순서를 모른다
마스크	소프트맥스 전에 -inf. 패딩은 키(열) 쪽, 인과는 tril. 함수마다 True 뜻이 반대
멀티헤드	view → transpose로 나누고 transpose → contiguous → view로 합친다. 파라미터 그대로

12.11 12장 정리 ②: 모델이 되기까지

부품	한 줄 요약
위치 정보	사인·코사인 또는 학습하는 위치 임베딩(nn.Embedding에 자리 번호)을 더한다
블록	x + 어텐션(층정규화(x)), x + 피드포워드(층정규화(x)). 블록 하나에 약 $12C^2$
인코더·디코더	인코더는 앞뒤 다(BERT), 디코더는 앞만(GPT), 교차 어텐션은 다른 문장의 키·값
작은 GPT	한 글자 밀린 조각이 정답. 첫 손실 -ln(1/어휘). 확률대로 한 글자씩. 문맥 비용은 제곱
실습의 교훈	손실이 너무 좋으면 누수를 의심한다. 인과 마스크가 빠지면 정답을 베낀다

인과 마스크를 뺀 모델은 검증 손실 0.056인데 왜 글이 망가졌을까요?

가 학습 데이터를 외워 과적합해서

나 학습 때 다음 글자를 볼 수 있어 베끼는 법을 배워서

다 위치 정보가 사라져서

라 1,000단계로는 학습이 부족해서

10차시 정리

실험은 **스위치 하나만** 바꾸고 같은 시드, 같은 단계로 비교한다

인과 마스크가 빠지면 손실은 0.05인데 글은 망가진다: **너무 좋은 손실은 누수부터 의심**

위치 임베딩이 없어도 인과 마스크가 순서 단서를 조금 주지만, **못 본 글에서는 있는 쪽이 낫다**

다음 차시(3일차): 한국어 문장을 자르는 법 · 토크나이저와 사전학습 인코더

2일차에 배운 것

1. 어텐션은 **부드러운 딕셔너리**, 점수는 √d로 나눈다
2. 셀프 어텐션은 순서를 모르고, **마스크**는 소프트맥스 전에 -inf
3. 멀티헤드 · 위치 정보 · 층 정규화 · 잔차로 **블록**을 조립했다
4. 헌법과 민법으로 **작은 GPT**를 처음부터 학습시키고 들여다봤다
5. 부품을 빼 보니 **너무 좋은 손실은 누수**였다

내일 3일차 (13장)

- 네이버 영화 리뷰와 한국어가 어려운 이유
- 글자·어절·형태소·서브워드 토큰화
- 임베딩과 처음부터 학습하는 기준 모델
- 사전학습 인코더 미세조정과 뉴스 제목 분류 실습